

Дополнительная лекция по Си № 5

28 апреля 2025 г.

Коновалов А.В.

Размеченные объединения в языке Си

Размеченное объединение или **тип-сумма** — составной тип данных, создаваемых на основе типов $T_1 \dots T_n$, такой, что элементы размеченного объединения представляют собой пары (i, d_i) , где i — номер варианта, а d_i — значение типа T_i . Мощность типа-суммы равна сумме мощностей множеств $T_1 \dots T_n$.

Есть языки программирования, где типы-суммы поддерживаются на уровне синтаксиса: Haskell (и другие ML-подобные функциональные языки), Rust (они там называются `enum`'ами) и т.д.

В языке Си нет поддержки размеченных объединений — есть просто объединения, которые «не помнят» тип сохранённого варианта:

```
union DL {  
    double d;  
    long l;  
};
```

```
union DL dl;  
dl.d = 3.14;
```

Если опишем такое объединение, то мы не будем знать, какое конкретно значение мы в нём сохранили.

Значит, чтобы описать размеченное объединение на Си, нужно явно добавить разметку. Как правило, разметку описывают при помощи целого числа-тега, метки представляют собой именованные константы. Старый стиль: описывать константы при помощи `#define`, рекомендуемый стиль: при помощи `enum`. Размеченное объединение в языке Си описывается как структура, содержащая поле тега и объединение значений.

Опишем размеченное объединение из вещественного числа двойной точности и длинного целого:

```
struct DL {
```

```

enum { DL_DOUBLE, DL_LONG } tag;
union {
    double d;
    long l;
} data;
};

```

При использовании размеченного объединения нужно согласованно присваивать тег и значение в объединении:

```

struct DL dl1, dl2;
dl1.tag = DL_DOUBLE;
dl1.data.d = 3.1416;
dl2.tag = DL_LONG;
dl2.data.l = 100500;

```

Следует помнить, что это лишь идиома, компилятор типы не проверяет, поэтому ответственность за согласованное изменение полей tag и data лежит на программисте.

Недостаток размеченных объединений — сложность добавления нового варианта: нужно не только добавить новый тег и новое поле в union, но и проверить всю программу, где происходит работа с ними и добавить логику для обработки нового варианта.

Для борьбы с этим недостатком придумали в своё время парадигму ООП с наследованием и полиморфизмом — можно добавлять к имеющейся программе новые разновидности имеющихся типов без изменения существующего кода: достаточно от имеющегося класса унаследовать потомка.

Иллюстрация размеченного объединения

Опишем геометрическую фигуру, которая может быть прямоугольником, кругом или треугольником и напишем функцию вычисления её площади:

```

#include <math.h>
#include <stdio.h>

struct Shape {
    enum { RECT, CIRCLE, TRIANGLE } tag;
    union {
        struct { double x, y; } rect;
        struct { double r; } circle;
        struct { double a, b, c; } triangle;
    } data;
};

double area(struct Shape *s) {
    double p, r;

```

```

switch (s→tag) {
    case RECT: return s→data.rect.x * s→data.rect.y;
    case CIRCLE:
        r = s→data.circle.r;
        return M_PI * r * r;
    case TRIANGLE:
        p = s→data.triangle.a + s→data.triangle.b + s→data.triangle.c;
        p = p / 2;
        return sqrt(p * (p - s→data.triangle.a)
            * (p - s→data.triangle.b) * (p - s→data.triangle.c));
}
}

```

Если мы захотим добавить новую геометрическую фигуру, то нужно добавить константу в enum, поле в union и ветку case в функцию area.

Имитация ООП в языке Си

Язык Си — не объектно-ориентированный язык, в нём нет синтаксических конструкций, которые выражают наследование и полиморфизм (инкапсуляция нас сегодня не интересует). Однако, как и в случае с размеченными объединениями, объектно-ориентированные средства можно в Си *имитировать*.

Согласно Стандарту, адрес первого поля структуры совпадает с адресом самой структуры:

```

#include <stdio.h>

struct C {
    double d;
    long l;
};

int main() {
    struct C c;
    printf("%p %p\n", &c, &c.d);
    return 0;
}

```

Если запустим, то увидим, что адреса одни и те же:

```
D:\Си>gcc example.c
```

```
D:\Си>a.exe
000000000061FE10 000000000061FE10
```

(Для объединений верно то, что адрес *каждого* поля объединения совпадает с адресом самого объединения).

Что там это свойство даёт? Равенство указателей даёт нам возможность безопасно преобразовать указатель на первое поле структуры в адрес самой структуры, если мы, конечно, уверены, что указатель указывает именно на первое поле структуры, а не на что-то иное.

```
struct C {
    double d;
    long l;
};

struct C c;
double *pd = &c.d;
struct C *pc = (struct C*) pd;
```

Приведение типа здесь безопасно — т.к. адреса совпадают.

Для примера бывают небезопасные приведения типа:

```
char *pc = ...;
int *pi = (int *) pc;
```

Это приведение небезопасно на платформах, которые требуют, чтобы указатели были выровнены, в частности, чтобы целые числа размещались только по адресам, кратным 4 (например, ARM). Указатель на char может ссылаться на любой адрес в памяти, поэтому его приведение к int * может породить некорректный указатель.

Преобразование указателя на первое поле структуры в указатель на саму структуру позволяет имитировать наследование в языке Си:

```
struct Base {
    int x, y;
};

struct Derived {
    struct Base base;
    int z;
};

struct Derived d;
struct Base *pb = &d.base;
struct Derived *pd = (struct Derived *)pb;
```

Для имитации наследования нужно первым полем «класса-потомка» сделать структуру «класса-предка», после чего можно будет безопасно как преобразовывать потомка в предка (путём получения адреса первого поля), так и наоборот (используя приведение типа).

Но, чтобы преобразовывать безопасно, нам нужно быть уверенным, что указатель на предок в действительности указывает на потомок. Проверку типа можно

осуществлять, например, введением тегов типов внутри базового класса:

```
struct Shape {
    enum { RECT, CIRCLE, TRIANGLE } tag;
};

struct Rect {
    struct Shape base;
    double x, y;
};

struct Circle {
    struct Shape base;
    double r;
};

struct Triangle {
    struct Shape base;
    double a, b, c;
};
```

Преимущество такого наследования по сравнению с размеченными объединениями: экономия памяти: для круга требуется лишь 16 байт памяти (4 на тег + 4 выравнивание + 8 на радиус). Размеченное объединение всегда требует 32 байта (тег + выравнивание + самое большое поле объединения).

Недостаток: тип во время выполнения изменить нельзя, в отличие от размеченного объединения.

Общий недостаток по сравнению с размеченным объединением — сложность добавления нового типа: нужно добавить новый тег, нового наследника и, что самое коварное (провоцирующее ошибки), добавить по всей программе обработку нового тега (добавить новые ветки case или else if для обработки нового тега).

Что нужно сделать? Обеспечить полиморфизм. Полиморфизм можно обеспечить использованием указателей на функцию. «Базовый класс» хранит указатель на функцию, при инициализации потомков устанавливаем соответствующую функцию.

Для примера опишем геометрические фигуры с операцией вычисления площади:

```
#include <math.h>
#include <stdio.h>

struct Shape {
    double (*area)(struct Shape *s);
};
```

```

double area(struct Shape *s) {
    return s->area(s);
}

struct Rect {
    struct Shape base;
    double x, y;
};

double rect_area(struct Shape *s) {
    struct Rect *r = (struct Rect*) s;
    return r->x * r->y;
}

void init_rect(struct Rect *r, double x, double y) {
    r->base.area = rect_area;
    r->x = x;
    r->y = y;
}

struct Circle {
    struct Shape base;
    double r;
};

double circle_area(struct Shape *s) {
    struct Circle *c = (struct Circle*) s;
    return M_PI * c->r * c->r;
}

void init_circle(struct Circle *c, double r) {
    c->base.area = circle_area;
    c->r = r;
}

struct Triangle {
    struct Shape base;
    double a, b, c;
};

double triangle_area(struct Shape *s) {
    struct Triangle *t = (struct Triangle *) s;
    double p = (t->a + t->b + t->c) / 2;
    return sqrt(p * (p - t->a) * (p - t->b) * (p - t->c));
}

```

```

void init_triangle(struct Triangle *t, double a, double b, double c) {
    t->base.area = triangle_area;
    t->a = a;
    t->b = b;
    t->c = c;
}

int main() {
    struct Rect r;
    init_rect(&r, 5, 6);

    struct Circle c;
    init_circle(&c, 10);

    struct Triangle t;
    init_triangle(&t, 12, 13, 5);

    printf("%f %f %f\n", area(&r.base), area(&c.base), area(&t.base));
    return 0;
}

```

Вывод программы:

```

D:\Си>a.exe
30.000000 314.159265 30.000000

```

В этом примере мы описали «полиморфную» функцию `area`, которая принимает указатель на фигуру и вычисляет её площадь, вызывая сохранённую функцию по указателю.

Для каждой из фигур мы описали структуру-«потомка», функцию вычисления площади и функцию-инициализатор. Последнюю — для удобства (можно было без неё обойтись).

Эта реализация уже лишена недостатка тегов типов — для того, чтобы расширить программу новым типом, достаточно описать нового «наследника» и функцию вычисления площади. Для удобства можно добавить и инициализатор.

Для примера добавим квадрат:

```

struct Square {
    struct Shape base;
    double x;
};

double square_area(struct Shape *sh) {
    struct Square *sq = (struct Square *) sh;
    return sq->x * sq->x;
}

```

```
void init_square(struct Square *s, double x) {  
    s->base.area = square_area;  
    s->x = x;  
}
```

Заметим, что чтобы добавить тип, нам не требуется менять существующий код, достаточно только добавить новый. Вот так можно изобразить ООП в языке Си.

Рекомендуемая литература:

- Axel-Tobias Schreiner. Object-Oriented Programming with ANSI-C — Hanser, 2011. — 223 p. ISBN 3-446-17426-5, скачать её можно с сайта автора: <https://www.cs.rit.edu/~ats/books/>